

Memory for rung-agent: Jev-Mem proposal

Rung has no memory that outlasts a session. Jev-Mem is a research project that gives an agent a graph memory steered by small, cheap decisions. The six open calls are now settled; this records what was decided and what gets built.

1. **The need is real and the fit is clean.** A new session starts blank. Memory plugs into two points of rung's turn loop, before the turn and after it.
2. **Decided: memory is a generic provider extension, off by default.** A provider is a separate MCP process. Jev-Mem is one provider and lives outside rung, in a container image in its own repository. A no-model baseline provider is the yardstick.
3. **Jev-Mem's headline results are not yet evidence for us.** Its default test harness picks the best of three answers using the gold answer, and its questions are written for chat about people, not for coding work.
4. **Smallest first slice costs nothing to run:** the provider seam, the baseline and two hooks, all offline. Live spend is capped: \$0.05 for a first check, then a \$10 hard cap, ledgered.

A graph memory where a small model answers the yes/no-style questions

Jev is a decision model: it is shown some text and a list of questions and returns a probability for each. Jev-Mem asks it only small questions. Ordinary code adds up the answers and enforces every limit.

Writing

- Each observation becomes one node, kept as raw text. Nothing is ever deleted or merged away.
- Two Jev calls label it and judge how it relates to up to ten similar memories.
- Cost: about two calls per write, roughly 0.2 s each.

Reading

- Jev decides which kind of link matters (meaning, time, cause, entity), how deep to look, and when evidence is enough to stop.
- A chat model then writes the answer from what was found.
- Cost: 3 to 16 Jev calls per question, capped by a call budget and a deadline.

Why it fits rung

- Jev-Mem calls the same endpoint rung already uses for its own decisions, with typed answers, recorded fixtures and cost reporting.
- Its split, "small questions asked, code composes the answer", is rung's own rule for judges.
- It is MIT licensed; a port needs the notice kept.

Verified

We read the code and tests, the licenses, the model catalog and rung's loop. Jev's measured speed (about 0.2 s) and price (about \$0.00004 per call) come from our own earlier probe.

Inferred, not run

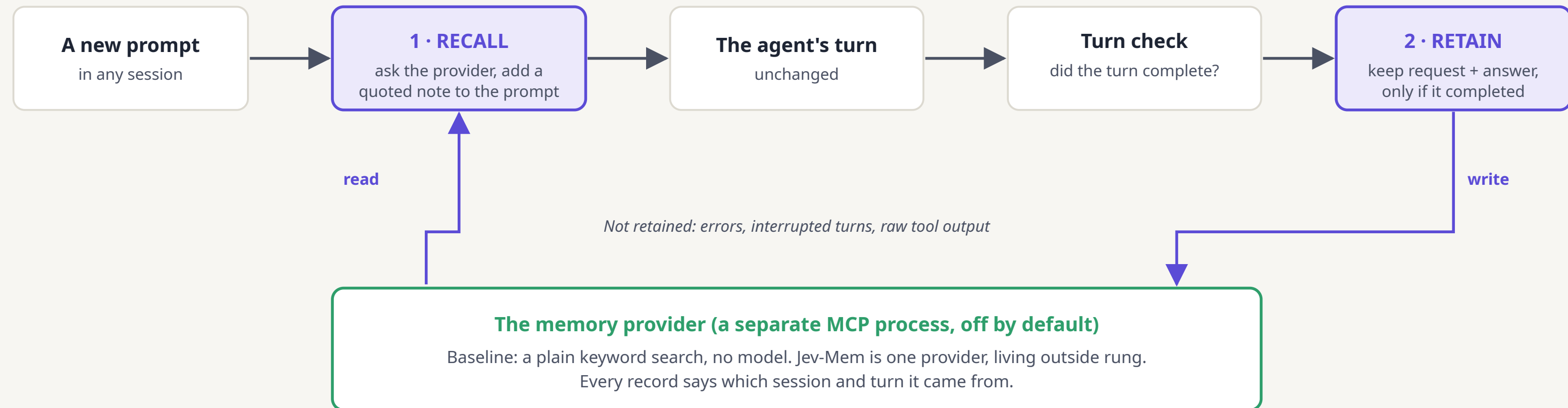
Jev-Mem was never run: its heavy packages (torch, faiss) are not installed and nothing was to be built. Per-turn latency of 0.6 to 4 s, cost estimates, effort estimates and the license-compatibility reading are our reasoning.

Only claimed

The paper reports 0.777 against 0.700 for its predecessor on one benchmark. We did not reproduce it, and found reasons to doubt it (page 4).

Two hooks around the turn, one provider, nothing else changes

Recall happens before the agent's turn and retention after it. The provider is a separate process behind a small, reserved interface, so it can be swapped.



Recalled text is shown to the model as quoted data from earlier sessions that may be wrong, never as an instruction, and it is never saved back into the session.

A generic provider seam in rung; Jev-Mem is one provider outside it

A provider is a separate MCP process. Rung stays small and depends on no provider, and Jev-Mem earns its place by measurement.

How a provider plugs in

- It is a separate MCP process that announces itself with a **rung-memory/1** marker.
- Two hook tools, **recall** and **retain**, are reserved: rung calls them itself, before and after a turn.
- Every other tool the provider offers is shown to the agent as usual.
- No provider configured means no memory, and rung opens no store of its own.

Why Jev-Mem stays outside

Its headline results are only claims: the default harness keeps the best of three answers using the gold answer, and its questions assume chat about people. It also needs a heavy Python environment, so that lives in a container image in a separate provider repository. Rung gets none of it. It must pass a test set written before the run: beat full replay and cheap retrieval, beat itself with Jev's control off, and add at most 3 s and \$0.005 per turn.

Smallest first slice: free and offline

W1. The shippable unit: the provider seam, the two reserved hooks and a no-model keyword provider, off by default. Tests prove a fact told in one session is recalled in another, and that with no provider nothing is stored.

W2. The Jev-Mem provider in its container, on mocked answers, behind the same seam.

W3. One tiny live check, at most \$0.05, through the accounted OpenRouter path.

W4. Five-way comparison (no memory, replay, keyword, Jev-Mem without Jev control, full Jev-Mem), \$10 hard cap, ledgered.

W5. Decide: port, keep the provider, or keep the baseline.

Safety throughout: retain only completed turns, never raw tool output, and show recalled text as quoted data.

The six calls are settled

Each answer was given by the owner; what it sets for the build is stated beside it.

1 Whose memory is it?

Decided: Per repository by default. The scope key is the project root, with no sharing across projects unless asked for.

2 Where may text go?

Decided: Anywhere: the owner does not mind the decision service seeing any text. It goes only through the accounted OpenRouter path, to the pinned decision model, with spend counted.

3 Spend

Decided: A first live check of at most \$0.05, then a \$10 hard cap, ledgered. Nothing beyond the cap runs.

4 Python footprint

Decided: It lives in a container image in a separate provider repository. It is never installed in the host's own Python, and rung carries none of it.

5 Benchmark data

Decided: Internal evaluation only, never committed. LoCoMo is CC BY-NC 4.0 and LongMemEval is MIT.

6 Memory in standalone rung?

Decided: Yes, as a generic provider extension that is off by default. The provider is a separate MCP process and Jev-Mem is one provider outside rung's core.

The main danger is a wrong note that becomes a trusted fact

Memory is persistent: whatever is stored can be recalled into every later session.

Risks and answers

- **Poisoned memory.** Keep only the user's request and the final answer of a completed turn. Never raw tool output. Show recalled text as quoted data.
- **Echo.** Strip the recall block before keeping a turn, and skip duplicates.
- **Secrets.** Redact key-like text before it is stored.
- **Hidden cost.** Jev-Mem silently falls back to an unbudgeted chat model on failure. The provider must turn that off and report each call's cost.

Jev-Mem as software

- Careful engineering: about 70 offline tests, validated settings, honest caveats.
- But it is research code: 18,000 lines, one week of history, and it stores data with Python's pickle format, which must only be loaded from our own files.
- Its questions assume chat about people. Rewording them for coding work voids the paper's tuning.
- Pin the model version; scores tuned on one version do not carry over.

The wider class

The real problem is state that must outlive the context window. Memory, trimming a session that grows too big and keeping a to-do list across prompts are one family. The longer-term fix is to separate three things: the **record** of what happened, the **memory** of what was selected, and the **assembly** of each prompt from both. This slice touches only memory.

Rung stays generic: it depends on no consumer, and a caller that owns memory simply configures no provider. A consumer was checked once to confirm that seam is enough.